Informatique *Cours*
Terminaison Ipesup

Objectifs

À l'issue de cette section, l'étudiant devra être capable de :
- Utiliser la techinique du variant pour démontrer la terminaison d'algorithmes.
- Comprendre les différentes structures de relations (équivalence, ordre, clôtures) et leurs propriétés fondamentales.
- Savoir comparer et organiser des éléments (notamment des paires) via des ordres produit et lexicographique.
- Utiliser les ordres bien fondés pour démontrer la terminaison d'algorithmes.

On s'attend la plupart du temps à ce qu'un programme, après avoir calculé un certain temps, renvoie un résultat ou achève la tâche qui lui avait été confiée, c'est-à-dire que son exécution *se termine*. Certains éléments en revanche ouvrent la possibilité pour l'exécution d'un programme de se poursuivre indéfiniment, sans jamais terminer. C'est le cas par exemple de la boucle `while` ou des appels récursifs de fonctions.

```
int i=1;
while (true) {
    printf("%ikm a pied, a use les souliers\n", i++);
}

let rec chanson n =
  Printk.printf "%ikm a pied, ca use les souliers\n" n;
  chanson (n+1)
in chanson 1
```

Lors de l'utilisation de boucles ou de fonctions récursives dans un algorithme dont l'exécution est supposée terminer, cet arrêt mérite donc une justification.

1 Technique du variant

L'algorithme d'exponentiation rapide en C (vu dans le cours precedente) utilise une boucle `while`, son exécution déclenche donc un nombre *a priori* inconnu de tours de boucle. On peut cependant tenir le raisonnement suivant : à chaque nouveau tour la valeur de n évolue pour devenir la partie entière inférieure de la moitié de la valeur précédente, c'est-à-dire un nombre entier positif strictement inférieur. Une telle décroissance ne pouvant avoir lieu indéfiniment sans que la valeur de n atteigne finalement zéro, la condition $n > 0$ de la boucle finira donc nécessairement par ne plus être vérifiée. L'exécution de la boucle s'arrêtera alors.

On peut tenir un raisonnement similaire sur la fonction OCaml, réalisant le même algorithme à l'aide de récursion. Un appel tel que `power x n` déclenche un nombre *a priori* inconnu d'appels récursifs à `power`, appliqués à différents entiers. On peut alors s'intéresser aux valeurs successives prises par le deuxième paramètre n et constater la même décroissance que dans la

fonction C. Le nombre d'appels récursifs emboîtés ne peut donc pas être infini, et la fonction finit bien toujours par renvoyer un résultat.

Dans le cas de la boucle comme dans celui de la fonction récursive, on a justifié la terminaison en identifiant une quantité qui *décroît*, et qui est de nature telle que cette décroissance ne peut se poursuivre indéfiniment. Cette quantité est appelée un variant.

Definition

Étant donnés un algorithme et une boucle de cet algorithme, un variant est une fonction des variables de l'algorithme qui :
- ne prend que des valeurs entières positives ou nulles,
- décroît strictement à chaque tour de boucle.

Étant donné un algorithme récursif, un variant est une fonction des variables de l'algorithme qui :
- ne prend que des valeurs entières positives ou nulles,
- décroît strictement à chaque appel récursif.

Notez que dans ces premiers exemples, nous avons directement utilisé l'une des variables manipulées par le programme comme variant pour justifier la terminaison. Cependant, le raisonnement fonctionne tout aussi bien avec une quantité qui serait calculée en fonction des différentes variables.

Exemple

Dans la recherche dichotomique étudié, aucune des variables du programme n'a la propriété de décroître strictement à chaque tour de boucle. D'une part la variable `lo` au contraire, lorsqu'elle évolue, croît. D'autre part, la variable `hi` a effectivement une tendance à la décroissance, mais est également susceptible de conserver sa valeur entre deux tours de boucles. Sa décroissance n'est donc pas stricte. En revanche nous savons qu'avant chaque nouveau tour de boucle, l'une des deux valeurs `lo` ou `hi` est modifiée, à la hausse ou à la baisse, de sorte que la différence $hi - lo$ décroisse à coup sûr. En outre, la condition d'arrêt de la boucle provoque l'arrêt du programme dès lors que cette différence devient nulle. Ainsi, nous pouvons justifier que ce programme de recherche dichotomique s'arrête à coup sûr en prenant comme variant la différence $hi - lo$ entre deux variables du programme.

Un phénomène similaire peut évidemment également être observé sur une fonction récursive, ainsi que dans des algorithmes ne manipulant pas directement des nombres.

Exemple

Voici par exemple une fonction OCaml réalisant la fusion de deux listes triées.

```
let rec merge l1 l2 = match l1, l2 with
| [], l | l, [] -> l
| x1 :: t1, x2 :: t2 ->
    if x1 <= x2 then
        x1 :: merge t1 l2
    else
        x2 :: merge l1 t2
```

Cette fonction ne manipule pas directement de nombres entiers, mais des listes. On peut

en revanche s'intéresser aux longueurs des listes manipulées, et justifier la terminaison de cette fonction par la décroissance de ces longueurs. Aucune des deux listes manipulées ne devient strictement plus courte à chaque nouvel appel ! L'une et l'autre décroissent à tour de rôle, selon une alternance *a priori* imprévisible. En revanche, il est bien acquis qu'à chaque nouvel appel, l'une des listes est plus courte, et l'autre égale. Ainsi, la somme des longueurs des deux listes, elle, décroît strictement à coup sûr, et cette quantité peut être prise comme variant.

Nous pouvons donc justifier la terminaison d'un programme en associant ses variables à une quantité décroissante. Cependant, pour certains programmes il peut être peu naturel voire impossible de se ramener à une valeur décroissante entière.

Nous allons donc développer la notion mathématique d'ordre, qui permet de décrire cette notion de décroissance dans un cadre plus général et donc aussi dans des programmes plus variés. Nous nous intéresserons en particulier aux ordres bien fondés : ceux pour qui toute décroissance se termine nécessairement, qui sont donc aptes à justifier la terminaison d'un programme.

2 Relations binaires et ensembles ordonnés

Une relation binaire entre les éléments de deux ensembles A et B est un ensemble d'associations entre un élément de A et un élément de B, caractérisant des éléments qui sont « en relation » l'un avec l'autre. La nature de cette « relation » peut couvrir des situations extrêmement variées. Par exemple :

- similarité entre objets, comme l'égalité $=$,
- hiérarchie entre un tout et ses parties, comme l'appartenance $\in$,
- comparaison de grandeurs, comme la comparaison $\leq$,
- antécédents et images d'une fonction,
- dépendance entre deux événements,
- incompatibilité ou interférence entre deux faits,
- accessibilité d'un point d'arrivée depuis un point de départ...

Certains de ces exemples correspondent à des types de relations importantes en mathématiques, à savoir les relations fonctionnelles, les relations d'équivalence et les relations d'ordre.

Définition

Soient A et B deux ensembles. Une relation entre A et B est un sous-ensemble de $A \times B$, c'est-à-dire un ensemble de couples d'un élément de A et d'un élément de B. Lorsque les ensembles A et B sont égaux, on parle de relation binaire homogène.
Étant donnée une relation $\mathcal{R} \subseteq A \times B$, et deux éléments $a \in A$ et $b \in B$, on note souvent $\mathcal{R}(a, b)$ ou $a\mathcal{R}b$ pour signifier $(a, b) \in \mathcal{R}$, c'est-à-dire que a et b sont en relation par $\mathcal{R}$.

En fonction de ses propriétés mathématiques, une relation binaire $\mathcal{R} \subseteq A \times B$ peut avoir différentes significations. Nous allons voir certains rôles majeurs que peut jouer une relation binaire et extraire les propriétés essentielles de chacun.

2.1 Relations fonctionnelles.

Une relation fonctionnelle est une relation binaire décrivant le lien entre les entrées et les sorties d'une fonction. Une telle relation $\mathcal{R}_f$ pour une fonction $f : A \to B$ contient donc les paires (a, b) telles que $f(a) = b$. Cet ensemble de paires est aussi appelé le *graphe* de la fonction f.

La caractéristique principale d'une telle relation, qui définit le concept de fonction, est que la sortie $f(a)$ est uniquement déterminée par l'entrée a. Autrement dit, il ne peut pas y avoir deux images associées au même antécédent.

Définition

Une relation $\mathcal{R} \subseteq A \times B$ est fonctionnelle si pour tout $a \in A$ il existe au plus un $b \in B$ tel que $a\mathcal{R}b$. Autrement dit, quels que soient $a \in A$ et $b_1, b_2 \in B$, si $a\mathcal{R}b_1$ et $a\mathcal{R}b_2$ alors $b_1 = b_2$.
Une relation fonctionnelle $\mathcal{R} \subseteq A \times B$ définit une fonction $f : A \to B$ de la manière suivante : pour tout $a \in A$, s'il existe un $b \in B$ tel que $a\mathcal{R}b$ alors $f(a)$ est définie et vaut b.

Notez qu'avec cette définition, une fonction $f : A \to B$ peut n'être que *partielle*, c'est-à-dire ne pas avoir de valeur $f(a)$ définie pour toute entrée $a \in A$. Le fait d'avoir un résultat défini pour toute entrée, appelé *totalité*, n'est que l'une des propriétés additionnelles que peuvent avoir des fonctions particulières.

Définition

Une fonction $f : A \to B$, définie par une relation fonctionnelle $\mathcal{R}_f \subseteq A \times B$, est :
- *totale* si pour tout $a \in A$ il existe un $b \in B$ tel que $a\mathcal{R}_f b$,
- *surjective* si pour tout $b \in B$ il existe un $a \in A$ tel que $a\mathcal{R}_f b$,
- *injective* si pour tous $a_1, a_2 \in A$ et $b \in B$, si $a_1\mathcal{R}_f b$ et $a_2\mathcal{R}_f b$ alors $a_1 = a_2$,
- *bijective* si elle est totale, surjective et injective.

Exemple

La fonction $f : \mathbb{N} \to \mathbb{N}$ définie par $f(n) = n^2$ a pour graphe l'ensemble $\{(n, n^2) \mid n \in \mathbb{N}\}$. Cette fonction est totale et injective. La même fonction étendue au domaine $\mathbb{Z} \to \mathbb{N}$ serait toujours totale mais plus injective, puisque $f(1) = f(-1) = 1$.

2.2 Relations d'équivalence.

Une équivalence regroupe les objets d'un ensemble en paquets en fonction de caractéristiques communes. Une telle classification est associée à une relation binaire homogène, qui associe deux à deux les objets appartenant à un même paquet. On isole trois propriétés d'une telle relation reflétant l'appartenance de plusieurs objets à une même classe :

- tout élément appartient à son propre paquet (*réflexivité*),
- l'énoncé « deux éléments appartiennent au même paquet », ne dépend pas de l'ordre dans lequel on considère les éléments (*symétrie*),
- l'appartenance à un même paquet se propage de proche en proche (*transitivité*).

Ces trois points réunis caractérisent la notion d'équivalence.

Définition

Une relation binaire homogène $\mathcal{R} \subseteq E \times E$ est :
- *réflexive* si pour tout $a \in E$ on a $a\mathcal{R}a$,
- *symétrique* si pour tous $a, b \in E$, si $a\mathcal{R}b$ alors $b\mathcal{R}a$,
- *transitive* si pour tous $a, b, c \in E$, si $a\mathcal{R}b$ et $b\mathcal{R}c$ alors $a\mathcal{R}c$.

Une relation d'équivalence est une relation binaire homogène qui est à la fois réflexive, symétrique et transitive.

Partant d'une relation binaire réflexive, symétrique et transitive, on peut reconstruire les « paquets » sous-jacents, appelés *classes d'équivalence*.

Définition

Soient E un ensemble et $\mathcal{R} \subseteq E \times E$ une relation d'équivalence sur E. La classe d'équivalence d'un objet $e \in E$ est l'ensemble

$$[e]_{\mathcal{R}} = \{e' \in E \mid e\mathcal{R}e'\}$$

des éléments qui sont en relation avec e. Si C est une classe d'équivalence pour la relation $\mathcal{R}$, tout élément $c \in C$ est un *représentant* de la classe C.

Exemple

Quel que soit son domaine de définition E, la relation d'égalité $=$ sur E est une relation d'équivalence, qu'on peut considérer comme la plus simple de toutes les relations d'équivalence. Ses classes d'équivalence sont tous les singletons de E.

Exemple

La relation $\mathcal{R} \subseteq \mathbb{N} \times \mathbb{N}$ définie par $x\mathcal{R}y$ si et seulement si x et y ont même reste modulo 3 est une relation d'équivalence ayant les trois classes suivantes.

$$[0]_{\mathcal{R}} = \{3k \mid k \in \mathbb{N}\}$$
$$[1]_{\mathcal{R}} = \{3k + 1 \mid k \in \mathbb{N}\}$$
$$[2]_{\mathcal{R}} = \{3k + 2 \mid k \in \mathbb{N}\}$$

Notez que $[0]_{\mathcal{R}} = [6]_{\mathcal{R}}$ ou encore que $[4]_{\mathcal{R}} = [1024]_{\mathcal{R}}$.

Les classes d'équivalence d'une relation d'équivalence $\mathcal{R}$ sur un ensemble E couvrent tout l'ensemble E sans redondance. Autrement dit, tout élément de E appartient à une et une seule classe d'équivalence. On dit que les classes forment une *partition* de l'ensemble.

Théorème de propriétés des classes d'équivalence

Soient E un ensemble et $\mathcal{R} \subseteq E \times E$ une relation d'équivalence sur E.
1. tout élément $e \in E$ appartient à au moins une classe d'équivalence,
2. si $a\mathcal{R}b$, alors $[a]_{\mathcal{R}} = [b]_{\mathcal{R}}$,
3. deux classes d'équivalence sont soit disjointes, soit égales.

Ces propriétés assurent que tout élément de E appartient à exactement une classe d'équivalence.

Démonstration

1. Tout élément $e \in E$ appartient à sa propre classe $[e]_{\mathcal{R}}$, car par réflexivité $e\mathcal{R}e$.
2. Soient $a, b \in E$ tels que $a\mathcal{R}b$. Montrons que $[a]_{\mathcal{R}} \subseteq [b]_{\mathcal{R}}$. Soit $c \in [a]_{\mathcal{R}}$. Par définition $a\mathcal{R}c$. Comme $a\mathcal{R}b$, par symétrie nous avons encore $b\mathcal{R}a$. Donc par transitivité $b\mathcal{R}c$, et ainsi $c \in [b]_{\mathcal{R}}$. Donc $[a]_{\mathcal{R}} \subseteq [b]_{\mathcal{R}}$, et l'inclusion réciproque est démontrée de même.
3. Soient deux classes $[a]_{\mathcal{R}}$ et $[b]_{\mathcal{R}}$ qui ne sont pas disjointes : on a donc un $c \in E$ tel que $c \in [a]_{\mathcal{R}}$ et $c \in [b]_{\mathcal{R}}$. De $c \in [a]_{\mathcal{R}}$ on déduit $a\mathcal{R}c$, et par le point précédent $[a]_{\mathcal{R}} = [c]_{\mathcal{R}}$. De même on déduit $[b]_{\mathcal{R}} = [c]_{\mathcal{R}}$, et finalement $[a]_{\mathcal{R}} = [b]_{\mathcal{R}}$.

2.3 Relations d'ordre.

Un ordre est une manière de décrire, dans un ensemble, des éléments qui sont « plus petits » ou « plus grands » que d'autres. Deux propriétés sont essentielles pour que cette comparaison ait un sens :

— on ne peut pas être à la fois plus petit et plus grand qu'un élément donné (*anti-symétrie*),
— la comparaison est transitive.

On a ensuite deux variantes possibles, selon que la relation est interprétée comme signifiant « strictement inférieur à » ou « inférieur ou égal à ». En considérant l'ordre usuel sur les nombres nous avons par exemple $1 \leq 1$ (comparaison large) mais pas $1 < 1$ (comparaison stricte).

Définition

Une relation binaire homogène $\mathcal{R} \subseteq E \times E$ est :
— *irréflexive* si aucun élément $a \in E$ ne vérifie $a\mathcal{R}a$,
— *anti-symétrique* si aucune paire d'éléments $a, b \in E$ tels que $a \neq b$ ne vérifie à la fois $a\mathcal{R}b$ et $b\mathcal{R}a$.

Un ordre est une relation binaire homogène qui est à la fois transitive, anti-symétrique et réflexive. Un ordre strict est une relation binaire homogène qui est à la fois transitive, anti-symétrique et irréflexive. Chaque ordre a un ordre strict associé, et réciproquement :
— si $\leq$ est un ordre, l'ordre strict associé est la relation $<$ définie par $a < b$ si et seulement si $a \leq b$ et $a \neq b$,
— si $<$ est un ordre strict, l'ordre (non strict) associé est la relation $\leq$ définie par $a \leq b$ si et seulement si $a < b$ ou $a = b$.

Si $\leq$ est un ordre, on note $\geq$ la relation telle que $x \geq y$ si et seulement si $y \leq x$.

Rien dans la définition des ordres ne demande que, lorsque l'on prend deux éléments, l'un des deux soit plus petit que l'autre. Un ordre ayant cette propriété est qualifié de *total*. Dans le cas contraire on parle d'*ordre partiel*.

Définition d'ordre total

Deux éléments $a, b \in E$ sont comparables pour un ordre $\mathcal{R} \subseteq E \times E$ lorsque l'on a $a\mathcal{R}b$ ou $b\mathcal{R}a$. Un ordre $\mathcal{R} \subseteq E \times E$ est :
— *total* si pour tous $a, b \in E$, a et b sont comparables.

Notez que cette notion de totalité est différente de celle qui s'applique aux fonctions : dans le cas des fonctions la totalité parle de l'existence d'une image pour tout élément de l'ensemble de départ, tandis que dans le cas des ordres elle parle de la possibilité de comparer toute paire d'éléments (dans un sens ou dans l'autre).

Exemple

La relation usuelle $\leq$ de comparaison entre deux nombres est un ordre total. L'ordre strict associé est la relation $<$.
La relation $\subseteq$ d'inclusion entre deux ensembles est un ordre partiel.

2.4 Clôtures.

Il est parfois pratique de décrire une relation en ne donnant qu'une partie de son graphe, mais en précisant que la relation doit être complétée pour avoir l'une des propriétés citées plus haut. On parle de *clôture* de la relation.

Définition

Soit $\mathcal{R} \subseteq E \times E$ une relation binaire homogène. La clôture transitive de $\mathcal{R}$ est la plus petite relation qui à la fois :
- contient $\mathcal{R}$, et
- est transitive.

On définit de même les clôtures réflexives, les clôtures symétriques, et leurs combinaisons. On note couramment $\mathcal{R}^+$ la clôture transitive d'une relation $\mathcal{R}$, et $\mathcal{R}^*$ sa clôture réflexive-transitive.

Notez que dans cette définition la notion de « plus petite relation » est à comprendre au sens de l'ordre d'inclusion, qui est bien applicable ici puisqu'une relation est par définition un ensemble (de paires).

Pour toute relation binaire homogène $\mathcal{R} \subseteq E \times E$, la clôture réflexive-symétrique-transitive de $\mathcal{R}$ est une relation d'équivalence. En revanche la clôture réflexive-transitive de $\mathcal{R}$ n'est un ordre que si la relation obtenue est bien anti-symétrique, et ceci n'est pas automatique.

Exemple

Notons S la relation de succession entre nombres entiers, dont le graphe est $\{(n, n+1) \mid n \in \mathbb{N}\}$. La relation d'ordre $\leq$ sur $\mathbb{N}$ usuelle est la clôture réflexive-transitive de S.
Si on considère en revanche la relation S_N de succession modulo N sur un intervalle d'entiers $[0, N[$, sa clôture réflexive-transitive $\mathcal{R}$ n'est pas un ordre. En effet, on a à la fois $(N-1)\ \mathcal{R}\ 0$, puisque $(N-1)\ \mathcal{R}_{S_N}\ 0$, et $0\ \mathcal{R}\ (N-1)$, puisque $0\ \mathcal{R}_{S_N}\ 1\ \mathcal{R}_{S_N}\ \cdots\ \mathcal{R}_{S_N}\ (N-1)$. La relation $\mathcal{R}$ n'est pas donc anti-symétrique.

3 Ensembles ordonnés

Un *ensemble ordonné* est un ensemble E auquel est associée une relation d'ordre $\leq \subseteq E \times E$, éventuellement partielle. L'ordre donne une certaine structure à l'ensemble.

3.1 Successions.

Une relation d'ordre $\leq$ sur un ensemble E permet d'identifier, pour un élément $e \in E$, les éléments qui sont plus petits ou plus grands que e.

Définition

Étant donné un élément e d'un ensemble ordonné E :
- un *prédécesseur* de e est un élément $a \in E$ tel que $a < e$,
- un *successeur* de e est un élément $a \in E$ tel que $e < a$.

Dans certains cas, l'ordre permet également d'identifier des éléments situés « juste avant » ou « juste après » un élément donné.

Définition

Étant donné un élément e d'un ensemble ordonné E :
- un *prédécesseur immédiat* de e est un élément $a \in E$ tel que $a < e$ et tel qu'il n'existe pas de $b \in E$ vérifiant $a < b < e$,
- un *successeur immédiat* de e est un élément $a \in E$ tel que $e < a$ et tel qu'il n'existe pas de $b \in E$ vérifiant $e < b < a$.

Exemple

- Dans $\mathbb{N}$ muni de l'ordre usuel sur les entiers, tout $n \neq 0$ admet $n - 1$ comme prédécesseur immédiat. En revanche, 0 n'a pas de prédécesseur, et donc pas de prédécesseur immédiat.
- Dans $\mathbb{R}$ muni de l'ordre usuel sur les nombres, aucun $e \in \mathbb{R}$ n'admet de prédécesseur immédiat. En effet, si on considère $e \in \mathbb{R}$ et $a < e$ un prédécesseur de e, alors on a toujours $a < \frac{a+e}{2} < e$.
- Considérons l'ordre d'inclusion $\subseteq$ sur les parties d'un ensemble E, et un ensemble non vide $X \in \mathcal{P}(E)$. Les prédécesseurs immédiats de X sont tous les ensembles obtenus en retirant exactement un élément à X.

3.2 Extrémaux.

Une relation d'ordre $\leq$ sur un ensemble E permet de comparer les éléments de E deux à deux, du moins pour ceux qui sont effectivement comparables. Ceci permet également d'identifier les éléments de E qui sont les plus petits. Pour cela cependant, deux critères sont possibles :

- être plus petit que tous les autres, ou
- être tel qu'aucun autre élément n'est plus petit.

Il se trouve que ces deux critères ne sont pas toujours équivalents. Le premier correspond à la notion de *plus petit élément*, et le second à la notion d'*élément minimal*. La même distinction existe également du côté des « plus grands ».

Définition

Considérant un ensemble ordonné E, et un élément $e \in E$:
- e est le *plus petit élément* de E si pour tout $a \in E$, $e \leq a$,
- e est le *plus grand élément* de E si pour tout $a \in E$, $a \leq e$,

— e est un *élément minimal* de E s'il n'existe pas de $a \in E$ tel que $a < e$,
— e est un *élément maximal* de E s'il n'existe pas de $a \in E$ tel que $e < a$.

Notez que l'on peut reformuler la définition des éléments minimaux ainsi : e est un élément minimal de E si pour tout $a \in E$ tel que $a \leq e$ on a $a = e$.

Exemple

Dans l'ensemble $\mathbb{N}$ des entiers naturels muni de l'ordre usuel sur les entiers, 0 est à la fois un plus petit élément et un élément minimal, et est le seul nombre à avoir l'une de ces propriétés. Il n'y a en revanche aucun plus grand élément ni aucun élément maximal.
Considérons l'ensemble $\mathcal{P}(\mathbb{N})$ des parties de $\mathbb{N}$, muni de l'ordre d'inclusion. L'ensemble vide $\varnothing \in \mathcal{P}(\mathbb{N})$ est un plus petit élément et un élément minimal, et est l'unique ensemble à avoir l'une de ces propriétés.
Considérons maintenant l'ensemble $\mathcal{P}(\mathbb{N})\backslash\{\varnothing\}$ des parties non vides de $\mathbb{N}$. Cet ensemble ne contient pas de plus petit élément : on peut par exemple constater qu'il n'existe aucun ensemble non vide qui soit plus petit à la fois que $\{0\}$ et que $\{1\}$. En revanche, cet ensemble contient une infinité d'éléments minimaux, à savoir tous les singletons $\{n\}$ pour $n \in \mathbb{N}$.

Propriétés des extrémaux

Considérons un ensemble E muni d'un ordre $\leq$.

1. Le plus petit élément de E, s'il existe, est unique.
2. Le plus petit élément de E est un élément minimal de E.
3. Si l'ordre est total et si e est un élément minimal de E, alors e est également le plus petit élément de E.

Démonstration.

1. Supposons que $a \in E$ et $b \in E$ vérifient tous deux la condition pour être le plus petit élément de E. On a pour tout $c \in E$, $a \leq c$. Comme $b \in E$ on en déduit $a \leq b$. De même, on a pour tout $c \in E$, $b \leq c$, dont on déduit $b \leq a$. Finalement, par antisymétrie de $\leq$, on obtient $a = b$: le plus petit élément est unique.
2. Supposons que a est le plus petit élément de E, et qu'un élément $b \in E$ vérifie $b < a$. Par définition du plus petit élément, $a \leq b$. Puisque $b < a$, on a également $b \leq a$ et par antisymétrie on obtient $a = b$, en contradiction avec $b < a$. Donc si a est le plus petit élément, alors b ne peut pas exister : a est un élément minimal de E.
3. Considérons un élément minimal e de E. Soit $a \in E$, montrons que $e \leq a$. L'ordre étant total, on a $a \leq e$ ou $e \leq a$. Dans le deuxième cas, la conclusion est immédiate. Supposons donc $a \leq e$. Ce cas est décomposé en deux sous-cas : $a = e$ ou $a < e$. Si $a = e$, on a bien $e \leq a$. Le cas $a < e$ est en revanche impossible, car il contredirait la minimalité de e.

3.3 Bornes.

L'ordre sur un ensemble E permet également de comparer les éléments de E à des sous-ensembles $A \subseteq E$. Les majorants et les minorants d'un ensemble A sont les éléments respectivement plus grands ou plus petits que tous les éléments de cet ensemble.

Définition

Considérant un sous-ensemble $A \subseteq E$ d'un ensemble ordonné E :
- un *minorant* de A est un élément $e \in E$ tel que pour tout $a \in A$, $e \leq a$,
- un *majorant* de A est un élément $e \in E$ tel que pour tout $a \in A$, $a \leq e$.

Notez qu'un minorant de l'ensemble A qui appartiendrait à A serait également le plus petit élément de A.

Les bornes d'un sous-ensemble A déterminent les limites de cet ensemble.

Borne inférieure et borne supérieure

Considérant un sous-ensemble $A \subseteq E$ d'un ensemble ordonné E :
- la *borne inférieure* de A est, s'il existe, le plus grand élément de l'ensemble des minorants de A,
- la *borne supérieure* de A est, s'il existe, le plus petit élément de l'ensemble des majorants de A.

Notez que si l'ensemble A admet un plus petit élément, celui-ci est également sa borne inférieure.

3.4 Monotonie.

Une fonction entre deux ensembles ordonnés est monotone si elle préserve l'ordre, c'est-à-dire si ses images préservent la comparabilité des éléments.

Définition

Soient $(A, \leq_A)$ et $(B, \leq_B)$ deux ensembles ordonnés et $f : A \to B$ une fonction. On dit que f est monotone si pour tous $a_1, a_2 \in A$ tels que $a_1 \leq_A a_2$ on a $f(a_1) \leq_B f(a_2)$.

Notez que cette notion générale de monotonie correspond à la définition d'une fonction numérique « croissante ». Attention au léger décalage de vocabulaire ici : dans le monde des fonctions numériques on a également une notion de fonction « décroissante », ainsi qu'une définition de la « monotonie » comme l'un quelconque des deux cas croissant ou décroissant.

Exemple

Considérons l'ensemble $(\mathbb{N}, \leq)$ des entiers naturels muni de l'ordre usuel, et l'ensemble $(\mathcal{P}(\mathbb{N}), \subseteq)$ des parties de $\mathbb{N}$ muni de l'ordre d'inclusion. Alors la fonction card : $\mathcal{P}(\mathbb{N}) \to \mathbb{N}$ associant à chaque ensemble fini son cardinal est monotone : pour tous deux ensembles A et B tels que $A \subseteq B$ on a bien $\text{card}(A) \leq \text{card}(B)$.

4 Ordres bien fondés

La notion d'ordre permet de donner du sens à la notion de « progression » dans le processus décrit par un algorithme : on considère avoir progressé dès lors qu'une certaine mesure sera devenue strictement plus petite vis-à-vis de l'ordre adéquat. En revanche, tous les ordres

n'empêchent pas une telle progression de se poursuivre indéfiniment. Cette propriété d'interdire toute décroissance infinie caractérise les ordres bien fondés.

Définition

Un ordre $\leq$ sur un ensemble E est bien fondé s'il n'existe pas de suite infinie strictement décroissante pour $\leq$. Autrement dit, en notant $<$ l'ordre strict associé à $\leq$, il ne peut pas exister de suite $(x_k)_{k\in\mathbb{N}}$ telle que pour tout $k \in \mathbb{N}$ on ait $x_{k+1} < x_k$.

Un ordre bien fondé est également appelé un *bon ordre*, et un ensemble muni d'un bon ordre un ensemble *bien ordonné*.

Exemple

L'ordre usuel $\leq$ sur l'ensemble $\mathbb{N}$ des entiers naturels est bien fondé, de même que l'ordre $\subseteq$ d'inclusion sur les parties d'un ensemble fini.
Par contre, les ordres suivants ne sont pas bien fondés :

- l'ordre usuel $\leq$ sur l'ensemble $\mathbb{Z}$ des entiers relatifs, car on peut descendre indéfiniment dans les négatifs,
- l'ordre usuel $\leq$ sur l'ensemble $\mathbb{R}^+$ des réels positifs, car on peut s'approcher indéfiniment de 0 sans jamais l'atteindre,
- l'ordre d'inclusion $\subseteq$ sur les parties d'un ensemble infini, car on peut définir une suite infinie d'ensembles qui ont toujours moins d'éléments mais restent infinis, comme la suite $([k, \infty[)_{k\in\mathbb{N}}$.

Les ordres bien fondés admettent également une caractérisation alternative, qui sera utile pour certaines preuves.

Caractérisation alternative des ordres bien fondés

Un ordre $\leq$ sur un ensemble E est bien fondé si et seulement si toute partie non vide de E admet un élément minimal.
Démonstration.
Sens direct. Supposons $(E, \leq)$ bien fondé et considérons $A \subseteq E$ une partie non vide. On raisonne par l'absurde : supposons que A n'admette pas d'élément minimal. Autrement dit, pour tout $a \in A$ il existe un $a' \in A$ tel que $a' < a$.
Comme A est non vide il existe au moins un élément $a_0 \in A$. Comme $\leq$ est bien fondé, il n'existe pas de suite infinie strictement décroissante à partir de a_0. Soit alors une suite $(a_k)_{k\in[0,N]}$ strictement décroissante d'éléments de A à partir de a_0, qui soit la plus longue possible. Comme $a_N \in A$ et A n'admet pas d'élément minimal, il existe $a_{N+1} \in A$ avec $a_{N+1} < a_N$. Donc $(a_k)_{k\in[0,N+1]}$ est une suite strictement décroissante dans A strictement plus longue que la précédente : contradiction.
Donc A doit admettre un élément minimal.
Sens retour. Supposons que toute partie non vide A de E admette un élément minimal. On raisonne par l'absurde : supposons que $(x_k)_{k\in\mathbb{N}}$ soit une suite infinie strictement décroissante dans E. On note A l'ensemble des valeurs de cette suite.
L'ensemble A est non vide : il contient par exemple x_0. Il admet donc un élément minimal x_k. Or $x_{k+1} < x_k$, avec $x_{k+1} \in A$, ce qui contredit la minimalité de x_k.
Donc il ne peut pas exister de suite infinie strictement décroissante dans E, et l'ordre $\leq$ est bien fondé.

4.1 Terminaison avec ordre bien fondé

Un ordre bien fondé peut donner un argument pour démontrer qu'un programme avec une boucle `while` ou une fonction récursive termine : il suffit d'identifier un ordre bien fondé d'une part, et des éléments du programme qui décroissent strictement selon cet ordre. Dans les cas les plus simples, on pourra se ramener à des valeurs entières et à l'ordre usuel $\leq$ sur $\mathbb{N}$. Parfois, il est cependant plus simple d'utiliser des ordres construits sur mesure, combinant plusieurs éléments du programme analysé. Certaines constructions usuelles sont connues pour produire des ordres bien fondés.

Comparer des paires d'entiers

Plutôt qu'aux simples entiers, intéressons-nous aux paires. Comment jugeons-nous les comparaisons suivantes ?

1. $(1, 2) < (3, 4)$?
2. $(1, 3) < (2, 4)$?
3. $(2, 3) < (1, 5)$?
4. $(1, 5) < (2, 3)$?

Selon le sens que l'on donne à la comparaison de deux paires d'entiers, certaines de ces propositions sont valides et d'autres non.

— Seule la première proposition vérifie le critère « tous les éléments de la première paire sont inférieurs à tous les éléments de la deuxième paire ».
— Les deux premières propositions vérifient le critère « chaque élément de la première paire est inférieur à l'élément correspondant de la deuxième paire ».
— Les trois premières propositions vérifient le critère « la somme des éléments de la première paire est inférieure à la somme des éléments de la deuxième paire ».
— Les propositions 1, 2 et 4 vérifient le critère « le premier élément de la première paire est inférieur à l'élément correspondant de la deuxième paire ».

Étant donnés deux ensembles ordonnés $(A, \leq_A)$ et $(B, \leq_B)$, nous allons définir dans cette section deux manières de définir un ordre sur les paires de $A \times B$ ayant de bonnes propriétés.

4.1.1 Produit cartésien

Un critère simple pour comparer deux paires consiste à demander que chaque composante de la première paire soit inférieure à la composante correspondante de la deuxième paire.

Ordre produit

On se donne deux ensembles ordonnés $(A, \leq_A)$ et $(B, \leq_B)$. L'ordre produit sur $A \times B$ est l'ordre $\leq$ tel que $(a_1, b_1) \leq (a_2, b_2)$ si et seulement si on a à la fois $a_1 \leq_A a_2$ et $b_1 \leq_B b_2$.

Selon cet ordre, on aurait $(1, 2) \leq (3, 4)$ et $(1, 3) \leq (2, 4)$, mais aucune comparaison entre $(1, 5)$ et $(2, 3)$. L'ordre produit n'est donc pas un ordre total.

Ordre produit bien fondé

Si $(A, \leq_A)$ et $(B, \leq_B)$ sont deux ordres bien fondés, alors l'ordre produit $\leq$ sur $A \times B$ est bien fondé.
Démonstration Raisonnons par l'absurde : supposons qu'il existe une suite infinie $(a_n, b_n)_{n\in\mathbb{N}}$ strictement décroissante pour l'ordre produit. On en déduit deux suites infinies décroissantes $(a_n)_{n\in\mathbb{N}}$ et $(b_n)_{n\in\mathbb{N}}$ pour les ordres $\leq_A$ et $\leq_B$. On a plus précisément, pour tout $n \in \mathbb{N}$, d'une part $a_n \geq_A a_{n+1}$ et $b_n \geq_B b_{n+1}$ et d'autre part au moins l'une des deux conditions $a_n \neq a_{n+1}$ ou $b_n \neq b_{n+1}$. Ainsi, au moins l'une des deux suites $(a_n)_{n\in\mathbb{N}}$ ou $(b_n)_{n\in\mathbb{N}}$ décroît strictement infiniment souvent. Cela contredit l'hypothèse selon laquelle l'ordre correspondant ($\leq_A$ ou $\leq_B$) est bien fondé.

Un ordre produit peut être utilisé pour justifier la terminaison d'un algorithme dans lequel plusieurs variables décroissent à tour de rôle.

Exemple

Reprenons la fonction `merge` de fusion de deux listes triées en OCaml. On peut associer à chaque appel `merge l1 l2` la paire (n_1, n_2) des longueurs des deux listes l_1 et l_2. On a alors décroissance stricte pour l'ordre produit à chaque appel récursif, puisque dans la première branche l'appel se fait avec la paire de longueurs $(n_1 - 1, n_2)$ et dans la deuxième branche avec $(n_1, n_2 - 1)$. Autrement dit, dans chaque appel récursif on a décroissance stricte sur une des deux composantes, et égalité sur l'autre. L'ordre produit sur des paires d'entiers naturels étant bien fondé, la fonction `merge` termine sur toute entrée.

4.1.2 Produit lexicographique

Un critère un peu plus élaboré consiste à comparer d'abord selon la première composante, puis de n'utiliser la deuxième composante que pour trancher en cas d'égalité sur la première. Ce mécanisme est à la base de la règle pour comparer deux mots du dictionnaire.

Définition

On se donne deux ensembles ordonnés $(A, \leq_A)$ et $(B, \leq_B)$. L'ordre lexicographique sur $A \times B$ est l'ordre $\leq$ tel que $(a_1, b_1) \leq (a_2, b_2)$ si et seulement si l'une des deux conditions suivantes est vérifiée :

$$\begin{cases} a_1 <_A a_2 \\ a_1 = a_2 \text{ et } b_1 \leq_B b_2 \end{cases}$$

Selon cet ordre, on aurait $(1, 2) < (1, 4)$ et $(1, 5) < (2, 3)$ mais pas $(2, 3) < (1, 5)$. L'ordre lexicographique est un ordre total : toutes deux paires peuvent être comparées, dès lors que les ordres $\leq_A$ et $\leq_B$ sont eux-mêmes totaux. Notez également que toutes deux paires comparables par l'ordre produit le sont encore pour l'ordre lexicographique, dans le même sens.

Ordre lexicographique bien fondé

Si $(A, \leq_A)$ et $(B, \leq_B)$ sont deux ordres bien fondés, alors l'ordre lexicographique $\leq$ sur $A \times B$ est bien fondé.

Démonstration

On utilise la caractérisation alternative des ordres bien fondés : toute partie non vide contient au moins un élément minimal.

Soit donc $C \subseteq A \times B$ un ensemble non vide arbitraire de paires d'un élément de A et d'un élément de B.

On note C_A l'ensemble des éléments de A apparaissant dans une paire de C. Formellement : $C_A = \{a \in A \mid \exists b \in B,\ (a, b) \in C\}$. Comme C n'est pas vide, C_A contient également au moins un élément. L'ordre $\leq_A$ étant bien fondé on en déduit qu'il existe un élément a_0 minimal pour $\leq_A$ dans C_A. Notons maintenant C_B l'ensemble des éléments de B apparaissant associés à a_0 dans l'ensemble C. Formellement : $C_B = \{b \in B \mid (a_0, b) \in C\}$. Cet ensemble C_B est à nouveau non vide, puisque $a_0 \in C_A$ et par définition de C_A il existe au moins un $b \in B$ tel que $(a_0, b) \in C$. L'ordre $\leq_B$ étant bien fondé on en déduit qu'il existe un élément b_0 minimal pour $\leq_B$ dans C_B.

Il ne reste plus qu'à montrer que la paire (a_0, b_0) est un élément minimal de C pour l'ordre lexicographique. Soit donc $(a, b) \in C$ telle que $(a, b) \leq (a_0, b_0)$. Par définition de l'ordre lexicographique $\leq$ on a deux cas.

- Soit $a <_A a_0$, ce qui contredirait la minimalité de a_0 car $a \in C_A$.
- Soit $a = a_0$ et $b \leq_B b_0$. Alors $b \in C_B$, et par minimalité de b_0 on a donc $b = b_0$.

On a donc nécessairement $(a, b) = (a_0, b_0)$, ce qui signifie que la paire (a_0, b_0) est bien minimale. Donc C admet nécessairement un élément minimal, et ainsi l'ordre lexicographique est bien fondé.

On peut utiliser un ordre lexicographique pour justifier la terminaison d'un algorithme dans lequel on a une hiérarchie entre les différentes variables susceptibles de décroître.

Exemple

Considérons la fonction `ack` prenant en paramètres deux entiers naturels et définie par les équations récursives suivantes.

$$\begin{cases} \texttt{ack}(0, m) = m + 1 \\ \texttt{ack}(n, 0) = \texttt{ack}(n-1, 1) \quad \text{si } n > 0 \\ \texttt{ack}(n, m) = \texttt{ack}(n-1,\ \texttt{ack}(n, m-1)) \quad \text{si } n > 0 \text{ et } m > 0 \end{cases}$$

Sa terminaison est immédiate en considérant l'ordre lexicographique sur les deux paramètres : chaque appel récursif implique soit une décroissance stricte sur le premier paramètre, soit une égalité sur le premier paramètre et une décroissance stricte sur le deuxième.

Notez que l'appel à un ordre lexicographique fonctionne alors même que l'un des appels récursifs est associé à une (très forte) croissance du deuxième argument : du moment où le premier argument décroît strictement, cela n'est pas interdit.

La fonction considérée ici, appelée *fonction d'Ackermann*, est connue entre autres choses pour le fait qu'il est impossible de prouver sa terminaison en se ramenant à un seul entier. L'ordre lexicographique est donc une forme de combinaison très puissante, qui permet de justifier la terminaison d'algorithmes que l'on n'aurait pas su traiter sans cela.

Ordre lexicographique sur des séquences non bornées

L'ordre lexicographique est également défini sur des séquences de tailles variables. C'est même précisément sous cette forme qu'il est utilisé pour ordonner les mots du dictionnaire !
Sous cette forme cependant, l'ordre lexicographique n'est pas bien fondé. En guise de contre-exemple, considérons pour chaque $k \in \mathbb{N}$ la liste OCaml l_k formée de k occurrences de 0 suivies d'une occurrence de 1. Ainsi on a par exemple $l_2 = [0;\ 0;\ 1]$ et $l_3 = [0;\ 0;\ 0;\ 1]$. Pour tout $k \in \mathbb{N}$ on a $l_{k+1} < l_k$. En effet, les deux listes commencent par k occurrences de 0, puis l_{k+1} poursuit avec un nouveau 0 là où l_k contient un 1. La séquence $(l_k)_{k\in\mathbb{N}}$ est donc une chaîne infinie strictement décroissante, qui tire parti du fait que l'on peut décroître selon l'ordre lexicographique avec des séquences de plus en plus longues. Cela démontre que l'ordre lexicographique sur des séquences non bornées n'est pas bien fondé.

4.1.3 Produit n-aire

Le produit cartésien et le produit lexicographique peuvent être généralisés à des n-uplets d'une taille fixée quelconque.

Définition

On se donne n ensembles ordonnés $(A_k, \leq_k)$. L'ordre lexicographique strict sur les n-uplets de $A_1 \times A_2 \times \cdots \times A_n$ est l'ordre $<$ tel que $(a_1, \ldots, a_n) < (b_1, \ldots, b_n)$ si et seulement s'il existe un k tel que $a_k <_k b_k$, avec pour tout $i \in [1, k[$, $a_i = b_i$.

On a défini ici l'ordre strict, donc la caractérisation est la plus simple. On peut en déduire l'ordre ordinaire $\leq$ en ajoutant le cas où les deux n-uplets sont égaux sur toutes leurs composantes.

Ordre lexicographique n-aire bien fondé

Si les $(A_k, \leq_k)$ sont des ordres bien fondés, alors l'ordre lexicographique $\leq$ sur $A_1 \times \cdots \times A_n$ est bien fondé.
Démonstration
Il suffit d'itérer la construction donnée dans la preuve du théorème pour l'ordre lexicographique bien fondé.

Exemple

On propose le principe suivant pour trier un tableau a de n entiers : tant que le tableau n'est pas trié, sélectionner deux indices i et j arbitraires tels que $i < j$ et $a[i] > a[j]$ et échanger les valeurs $a[i]$ et $a[j]$. Sans plus de précision, le fonctionnement exact d'un tel tri est imprévisible. Sans une stratégie intelligente pour détecter et choisir les paires à inverser, il risque même d'être très inefficace. Cependant, nous avons déjà assez d'éléments pour justifier que la procédure termine à coup sûr après un nombre fini d'inversions.
Considérons l'ensemble du tableau de taille n comme un n-uplet $a = (a_1, \ldots, a_n)$ d'entiers, et prenons l'ordre lexicographique sur $\mathbb{N}^n$. Après l'inversion de deux éléments a_i et a_j avec $i < j$ et $a_i > a_j$ on obtient un n-uplet $a' = (a'_1, \ldots, a'_n)$ tel que :

$$\begin{cases} a'_i = a_j \\ a'_j = a_i \\ a'_k = a_k \quad \text{si } k \neq i \text{ et } k \neq j \end{cases}$$

Pour tout $k \in [1, i[$ nous avons donc $k \neq j$ et $a'_k = a_k$. En outre, $a'_i = a_j < a_i$. Donc $a' < a$ pour l'ordre lexicographique sur les n-uplets, et le tri terminera bien nécessairement après un nombre fini d'inversions.

5 Relations non binaires

Nous nous sommes concentrés ici sur les relations binaires, caractérisant des paires d'éléments. La notion de relation peut être généralisée à des n-uplets quelconques.

On peut par exemple considérer la relation ternaire P qui identifie les triplets d'entiers (a, b, c) vérifiant $a^2+b^2 = c^2$ (on les appelle des *triplets pythagoriciens*, ils font référence aux longueurs des côtés d'un triangle rectangle). Pour cette relation, on a par exemple $P(3, 4, 5)$, ou encore $P(48, 55, 73)$.

Le cas d'une relation unaire sur un ensemble E correspond, lui, directement à un sous-ensemble d'éléments E.